

RSA Signatures - Cryptohack

Cryptography

Analysis

deskripsi chall

How can you ensure that the person receiving your message knows that you wrote it?

You've been asked out on a date, and you want to send a message telling them that you'd love to go, however a jealous lover isn't so happy about this.

When you send your message saying yes, your jealous lover intercepts the message and corrupts it so it now says no!

We can protect against these attacks by cryptographically signing the message.

Imagine you write a message m . You encrypt this message with your **friend's public key**: $c = m \cdot e \bmod N$. $c = m \cdot e \bmod N$.

To sign this message, you calculate the hash of the message: $H(m)$ and "encrypt" this with **your private key**: $s = H(m) \cdot d \bmod N$. $s = H(m) \cdot d \bmod N$.

In real cryptosystems, it's [best practice to use separate keys](#) for encrypting and signing messages.

Your friend can decrypt the message using **their private key**: $m = c \cdot d \bmod N$. $m = c \cdot d \bmod N$. Using your public key they calculate $s = e \cdot s \bmod N$. $s = e \cdot s \bmod N$.

Now by computing $H(m)$ and comparing it to s : `assert H(m) == s`, they can ensure that the message you sent them, is the message that they received! As long as your private key is safe, no one else could have signed this message!

Sign the flag `crypto{Immutable_message}` using your private key and the `SHA256` hash function.

The output of the hash function needs to be converted into a number that can be used with RSA math. Remember the helpful `bytes_to_long()` function that can be imported from `Crypto.Util.number`.

Challenge files:

- [private.key](#)

di deskripsi chall kita di kasih kasus dimana kita di ajak date, dan kita sangat pengen menerimanya, tetapi pasangan kita orang yang cemburuan (padahal udah punya pasangan tapi masih mau ngedate sama yang lain 🤨). ketika kita mengirim pesan “yes” pasangan kita yang cemburuan itu merubah nya jadi “no”.

jadi solusi nya kita mengenkripsi pesan kita menggunakan kunci publik teman kita supaya dia bisa mendekripsikan ct_m jadi plaintext, kenapa hal tersebut bisa terjadi? karena sifat rsa yang dimana

$$(M^{**e})^{**d} \bmod n == (M^{**d})^{**e} \bmod n == m \bmod n$$

jadi ketika kita mengenkripsi M menggunakan d maka bisa mendekripsikan ct_M menggunakan kunci e

.nah untuk memastikan bahwa pesan yang kita kirim itu benar benar dari kita, maka kita harus membuat “patokan” untuk memverifikasi bahwa pesan itu benar benar dari kita.

Solusi

jadi pertama kita hash plain dari pesan kita setelah itu kita enkripsi hash_m menggunakan kunci privat kita supaya teman kita bisa mendeskripsi ct_hash_m kita menggunakan kunci publik (karena keunikan RSA) dan hasil dari dekripsi tadi adalah hash_m kita, karena hash itu sifat nya 1 arah maka nilai hash itu adalah patokan. jika pesan kita di ubah oleh pasangan kita maka jika pesan kita di rubah dan di jadikan hash maka akan terjadi efek berantai dan jika di bandingkan dengan hash kita hasil nya tidak identik, maka bisa di simpulkan bahwa pesan tersebut sudah dirubah.

solver =

```
from hashlib import sha256

p = 'crypto{Immutable_m3ssag1ng}'
n =
15216583654836731327639981224133918855895948374072384050848479908982286
89073176948660908591885766404607537525316895505874318566439027305807445
03902367743249033056634790465662329672977657316253280298140556353160025
91227570271271445226094919864475407884459980489638001092788574811554149
77402895031069511268872385376374323875334978250812198533874675523781937
31786993431350917839922995618273897451328800222598733875242732988503406
48779897909381979714026837172003953221052431217940632552930880000919436
50724515072654304071472155336106331195428528985758207988029519963275782
9525723874753306371990452491305564061051059885803
d =
11175901210643014262548222473449533091378848269490518850474399681690547
28166505931715583169230045319733573572845925939236682330240568538958688
36700437446839937091231808051546310885135214569793176280127218815371541
07239389466063136007337120599915456659758559300673444689263854921332185
```

```
56270670757366065816499109845787449505485449147406503962192297267158829
93158463060698451699594512508210444178866303462290213054103401004015301
46135418806544340908355106582089082980533651095594192031411679866134256
41829224959213544114538446626127942879540872199056465870390378795695816
8449841491667690491585550160457893350536334242689
```

```
object_sha256 = sha256(p.encode())
```

```
h_m2 = object_sha256.hexdigest()
```

```
c = pow(int(h_m2,16),d,n)
```

```
print(c)
```